Model Compression for Resource-Constrained Federated Learning Using Knowledge Distillation

Vsevolod Kharchenko

February 8, 2026

Abstract

Federated learning (FL) is a powerful model training method in which a model is trained on multiple devices instead of a single device with a centralized database. Its decentralization also opens up new possibilities in privacy-preserving machine learning. The problem, however, is that communication bandwidth of edge networks and the hardware's computational power are quite limited in real life scenarios. This paper proposes a multi-stage model compression pipeline for efficient resource-constrained federated learning through the use of response-based Knowledge Distillation, Post-Training Quantization, Adaptive Unstructured Pruning, and through the implementation of FedDF - Federated Distillation Fusion. This pipeline will allow users to compress a model heavily enough to fit the most resource-constrained FL scenarios.

1 Introduction

Due to the spread of IoT and smart edge devices, large amounts of decentralized data, useful for Deep Learning (DL) and other machine learning (ML) applications, have appeared. Centralized storage and collection of user data, which is common when working with ML/DL, is problematic due to bandwidth limitations, many regulations (e.g. GDPR) and general privacy concerns.

Federated Learning (FL) aims to address this: instead of collecting the data and bringing it to one, centralized server for model training, it instead sends the model to data, eliminating the issue of centralization. The problem is, however, that the models are usually too computationally expensive and storage-heavy for most smart edge devices, thus not solving the bandwidth issue (e.g. 20 rounds of training for a standard 42 MB ResNet-18 involves transmitting around 2 GB of data per client).

This study addresses these challenges through a three-stage analysis:

1. Model Compression: Model compression methods (Pruning, Quantization, and Knowledge Distillation) are compared and analyzed to pick the best option for an efficient client-side "Student" model.
2. Edge Deployment: The best models are converted to TensorFlow Lite format and, with additional quantization, tested on a CPU to measure the inference efficiency of the models.
3. Scalable Federation: A heterogeneous FL environment is simulated, where clients deploy adaptively pruned and quantized models, aggregated via Federated Learning.

Our main contribution is proving that shifting from weight-averaging (FedAvg) to knowledge-averaging (FedDF) with compressed models allows for large savings in bandwidth with manageable accuracy trade-offs.

2 Related Work

2.1 Federated Learning

First introduced by McMahan et al [1], FedAvg serves as the canonical baseline. The problem is that it assumes homogeneity across client model architectures, which makes methods such as Knowledge Distillation unviable.

2.2 Model Compression

Pruning and Quantization are well-established and widely used model compression techniques [2]: Pruning removes redundant weights (Han et al. [3]), and Quantization reduces precision (e.g., Float32 to Int8), resulting in immediate size and speed benefits on modern CPUs (Jacob et al. [4]).

2.3 Knowledge Distillation in FL

Hinton et al. [5] proposed distilling a "Teacher" into a "Student." In FL, this has been adapted (e.g., FedDF) to allow clients to train on local data and share "knowledge" (logits) on a public proxy dataset, rather than sharing weights.

3 Methodology

3.1 Dataset and Preprocessing

We choose the CIFAR-10 dataset [6]. It contains 60,000 color images with a resolution of 32 x 32 pixels, and with 10 classes (e.g., airplanes, birds, cats, etc.), to evaluate our methods on. It is split into 50,000 training samples and 10,000 testing samples.

Standard data augmentation techniques are applied to the training data with the goal of improving model generalization: random horizontal flipping and random cropping with padding of 4 pixels. All images are also normalized using the dataset's channel-wise mean and standard deviation parameters.

The testing set is split into proxy and evaluation sets: 33.3% of the 10000 test images go into the proxy set used for server-side distillation fusion, while the rest goes into the evaluation set. The split is done to prevent data leakage during the evaluation, and to simulate what would be public data available to the server in a real-world scenario.

We also split 50,000 training samples into 5 sets, one for each client during FL simulation, with an Independent and Identically Distributed (IID) strategy. This way, each client has a balanced 10,000 image dataset, simulating a scenario where devices have roughly equivalent data volume and feature distribution.

3.2 Model Architectures

For our baseline, we use **ResNet-18** (11.6M parameters), a standard architecture for CIFAR-10 classification. For our purposes, we modified the first convolution layer of the model to avoid downsampling the already small images from CIFAR-10 too quickly.

For knowledge distillation, we design a custom student network, **BiggerStudent**:

- Structure: 3 sequential blocks. Each block consists of 2 batch normalized and ReLU-activated convolution layers with kernel size = 3, and padding = 1, and a 2d Max Pooling layer. After the 3 blocks, Global Average Pooling is performed, followed by a linear layer at the end.
- Rationale: much shallower than ResNet-18 while retaining the ability to learn distilled features.

Along with that, a small DistilledStudent is defined: 3 convolution layers with kernel size = 3 and padding = 1, each followed by a batch normalization layer, activated, and pooled with Max Pooling, then projected onto a linear layer. It exists purely to test the most extreme compression.

3.3 Compression Techniques

To compress our baseline model's knowledge into the BiggerStudent, we employ Response-Based Knowledge Distillation with a loss function:

$$L_{KD} = \alpha T^2 \cdot D_{KL}(p_s||p_t) + (1-\alpha) \cdot L_{CE}(y, p_s)$$

Where D_{KL} is the Kullback-Leibler divergence between the student (p_s) and teacher (p_t) soft targets (temperature $T = 4.0$ to transfer dark knowledge), and L_{CE} is the standard cross-entropy loss against true labels. Simply put, we train the student both on labeled data and on the teacher's softened outputs. This way it learns the connections between classes previously learned by the teacher.

In a realistic scenario, very different client devices may be used for FL. Some may be able to handle larger models, some may not. In order to simulate specific client needs, we implement Adaptive Unstructured Pruning: we assign weight removal targets to different clients (e.g. 20% to client 1, 50% to client 2) and remove the assigned amount of weights through L1-norm magnitude pruning.

Quantization for federated learning simulations is done via AI Edge Torch. We use Dynamic Range Quantization (INT8): we store weights as 8-bit integers, and quantize activations during inference. However, we do not fully implement INT8 quantization during FL simulation, we simulate INT8 transmission by calculating the payload size as Size = $N_{non-zero} \times 1$ byte during bandwidth savings calculation instead.

3.4 Federated Strategies

FedAvg will serve as our baseline. The server and 5 clients each have their own pre-trained ResNet-18. First, the clients train their models, then the weights are sent back to the server and averaged: $w_{global} = \sum \frac{n_k}{n} w_k$. Communication cost is calculated as the full FP32 model size per upload.

As mentioned earlier, FedAvg assumes homogeneity, which is not possible with KD. We utilize FedDF (Federated Distillation Fusion) [7] to solve this issue:

1. Client phase: clients are initialized with a pre-trained BiggerStudent and a local dataset. They train locally using the same loss function as defined earlier for knowledge distillation with temperature T = 2. After training, each client prunes its model by removing the percentage of smallest weights equal to the assigned weight removal target(M_k).

2. Server phase: the server has a pre-trained teacher model (ResNet-18) and a proxy dataset. After the client phase, the server computes average client logits (ensemble knowledge) on the proxy set and updates its model to match the ensemble knowledge with KL Divergence loss.

4 Experimental Setup

The model training is done using an Nvidia RTX 5080 GPU with PyTorch. Compression, evaluation and benchmarking was performed on an AMD Ryzen 9 9900x CPU with PyTorch and TensorFlow Lite. Preliminary stage 1 training and testing, however, is performed on an Nvidia GTX 1660 super GPU and AMD Ryzen 5 3600 CPU.

The dataset used is the CIFAR-10 dataset: 60000 images, 5:1 train-test split, augmented with random cropping and random horizontal flipping:

- Baseline/students training: train set split into 15% validation set and 85% training set.

- FL: test set split 1:2 into proxy and evaluation sets, train set split into 5 client sets(IID). Batch size = 512 for data loading.

The baseline ResNet-18 is trained on the CIFAR-10 dataset for 35 epochs, with an SGD (Stochastic Gradient Descent) optimizer, CE (Cross-Entropy) loss function, and torch CosineAnnealingLR Learning rate scheduler.

The BiggerStudent model is trained with the response-based Knowledge Distillation (KD) approach as described by Zhuo Li et al [2]. The stage 1 pre-training was performed for 50 epochs with T = 4.0, SGD optimizer, KL Divergence + CE loss as defined earlier, and torch CosineAnnealingLR Learning rate scheduler. Baseline ResNet-18 serves as the teacher, the soft target loss weight is set to 0.9, and CE loss weight = 0.1. DistilledStudent is trained the same way except with soft target loss = 0.75, and CE loss = 0.25.

FedAvg has 5 clients and 1 server, all with pre-trained ResNet-18 models. FedDF has 5 clients with pre-trained BiggerStudents and 1 server with a pre-trained ResNet-18. Both methods are performed for 20 rounds with T = 2.0, client learning rate = 0.005 (SGD) and server LR = 0.0001. FedAvg clients perform 3 training epochs, FedDF clients perform 5 epochs. The α parameter for server distillation is set to 0.5.

5 Experiment Results

Our proposed pipeline is evaluated in three stages: model compression technique analysis, edge deployment simulation, and scalability assessment in a simulated FL environment.

5.1 Stage 1 - Compression Analysis

Unstructured L1 Pruning, Post-Training INT8 Quantization (PTQ), and Knowledge Distillation (KD) are evaluated on the CIFAR-10 dataset to determine the most effective compression method for the client model. The performance trade-offs are visualized in Figure 1 and summarized in Table 1.

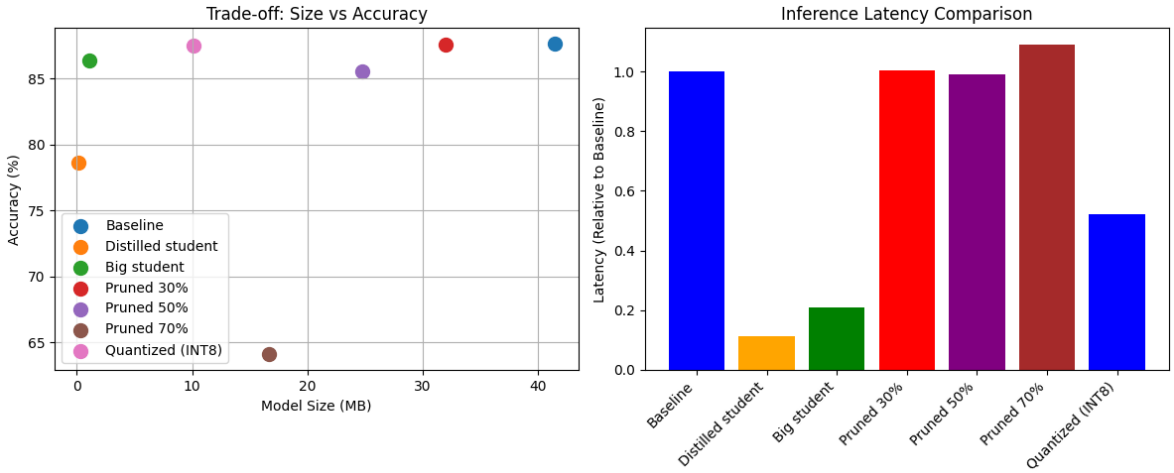


Figure 1: Stage 1 Trade-off Analysis: Model Size vs. Accuracy vs. Latency. The Distilled 'BiggerStudent' (Green) offers the best balance of size reduction and accuracy retention.

Table 1: Comparison of Compression Techniques (Stage 1)

Model	Size (MB)	Acc (%)	Latency (ms)
Baseline (ResNet-18)	41.51	87.63	940.31
Pruned 30%	31.99	87.59	943.63
Pruned 50%	24.74	85.51	929.90
Pruned 70%	16.68	64.10	1025.60
Quantized (INT8)	**10.15**	**87.50**	**489.59**
Distilled (DistilledStudent)	0.13	78.59	104.52
Distilled (BiggerStudent)	**1.08**	**86.37**	**196.97**

Pruning: Despite successfully reducing model size, it comes at the cost of a significant accuracy drop and no reduction in latency. The latter is due to the removed weights still participating in computations as zeros. 70% pruning even resulted in a catastrophic accuracy drop down to 64.10%.

Quantization: PTQ (INT8) resulted in a 4× size reduction with negligible accuracy loss. Along with that, a 2× speedup occurred (latency dropped from 940ms to 490ms).

Distillation: The BiggerStudent architecture achieved a 38× size reduction (1.08 MB) and 4.7× speedup compared to the baseline, with only a 1.26% drop in accuracy. The more extreme DistilledStudent resulted in a massive 319× size reduction and the fastest inference of 104.52 ms, but at the cost of a large accuracy drop.

Quantization is the safest method out of all tested, resulting in a reliable 4× size drop and a 2× speedup at a negligible accuracy loss, however BiggerStudent outperformed quantization at only a 1.26% accuracy drop, making KD the winner in this test. DistilledStudent achieves extreme compression, but the accuracy drop is too great to make this model viable. Due to the accuracy loss of DistilledStudent, BiggerStudent is chosen as the base architecture for the next stages.

5.2 Stage 2 - Edge Deployment Benchmarks

To validate real-world feasibility, selected models were converted to TensorFlow Lite and benchmarked on an AMD Ryzen 9 9900x CPU. We evaluated Dynamic Range Quantization (INT8) and Float16 (FP16) quantization. Figure 2 illustrates the Inference Efficiency Frontier.

In this stage, we only perform CPU operations. We convert the BiggerStudent and our baseline into TensorFlow Lite format and benchmark them. We evaluate Float16 (FP16) Quantization and Dynamic Range INT8 Quantizaiton. This will let us see the real-world feasibility of the compressed models. Figure 2 shows the Inference Efficiency Frontier and the energy-latency regression. More detailed comparison between models can be seen in Table 2.

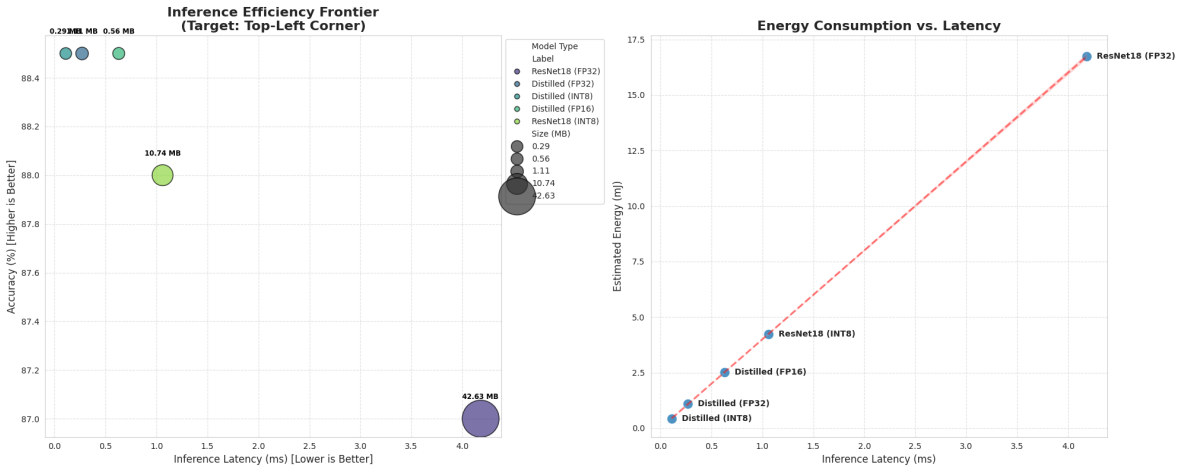


Figure 2: Stage 2 Edge Deployment: (Left) Inference Efficiency Frontier showing the INT8 Distilled model in the optimal top-left corner. (Right) Energy consumption vs. Latency.

Table 2: Benchmarking Results on CPU (Stage 2)

Model	Size (MB)	Accuracy (%)	Latency (ms)	Mem. Peak (MB)	Est. Energy (mJ)
ResNet-18 (FP32)	42.63	87.0	4.01	0.12	16.05
BiggerStudent (FP32)	1.11	88.5	0.26	0.11	1.05
BiggerStudent (INT8)	0.29	88.5	0.10	0.11	0.41
BiggerStudent (FP16)	0.56	88.5	0.62	0.11	2.47
ResNet-18 (INT8)	10.74	88.0	1.02	0.11	4.10

ResNet-18: The baseline model, quantized or not, is inferior to every other model among all tested in every category. INT8 quantization achieves great compression down to 10 MB, along with a 4× decrease in latency and estimated energy, at seemingly no cost, but the model remains inferior to the BiggerStudent.

BiggerStudent: The distilled model handles the compression and conversion very well, maintaining high accuracy after quantization. The INT8 quantized BiggerStudent outperforms every other tested model with a dramatic size reduction and best results in every other metric. It's 147× smaller, 1.5% more accurate, and 40× faster than the baseline.

However, FP16 BiggerStudent is showing strange results: while it achieved a drop in size and maintained accuracy, its latency and energy cost grew. This is most likely due to the lack of native FP16 support on CPU: the weights are de-quantized during processing, creating additional costs.

5.3 Stage 3 - Federated Scalability

This stage is where we test the compressed models in a simulated FL scenario. We simulate FedDF with 5 clients with 20%, 30%, 40%, 50%, and 50% adaptive pruning rates, and standard FedAvg. FedDF uses ResNet-18 on server and INT8-quantized BiggerStudents on each client, while FedAvg uses ResNet-18 for clients and server. The results are visualized in Figure 3 and in Table 3.

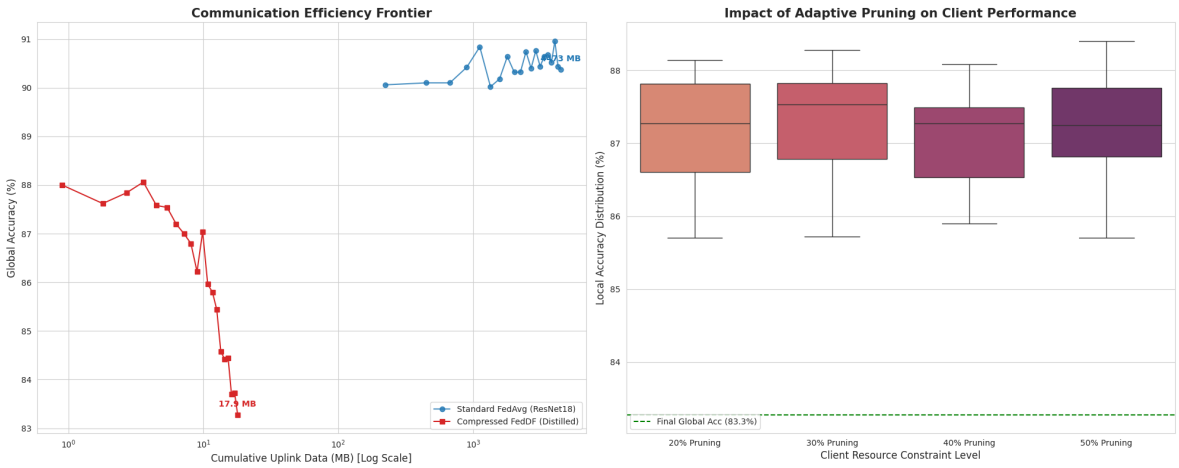


Figure 3: Stage 3 Federated Scalability: (Left) Communication Efficiency Frontier showing 99.6% bandwidth savings. (Right) Impact of Adaptive Pruning on Client Performance, showing robustness even at 50% pruning.

Table 3: Final Comparison - Standard FedAvg vs. Compressed FedDF (Stage 3)

Metric	**Standard FL**	**Compressed FedDF**
Final Accuracy	90.38%	83.28%
Total Comm.	4473.44 MB	17.94 MB
Comm. Savings	**99.60%**	

5.3.1 Communication Efficiency

The left plot of Figure 3 plots the global accuracy against cumulative uplink data transferred (log scale):

- **FedAvg:** Achieved a peak accuracy of 90.38% but required transmitting 4473.44 MB of data over 20 rounds.
- **FedDF:** Achieved 83.28% accuracy with a total communication cost of only 17.94 MB.

Using FedDF results in a 99.60% reduction in communication bandwidth at the cost of a 7.1% accuracy drop. The low accuracy is concerning, but the extreme reduction in communication bandwidth makes FedDF possible on networks where transmitting gigabytes of data is impossible (e.g. 2G/3G, NB-IoT, Bluetooth 3.0+).

5.3.2 Stability and Heterogeneity

Looking at the right plot in Figure 3, we see that clients do not degrade and maintain high accuracy even after 50% pruning. This shows that the performance drop seen on the left plot of Figure 3 is not due to client degradation, and tells us that the BiggerStudent architecture is robust to pruning and that adaptive compression is viable for handling heterogeneous devices. The drop in server model's accuracy likely happens due to the difficulty of distilling a consensus from diverse, pruned models.

6 Discussion

The experimental results across the three stages validate the proposed pipeline as a robust framework for resource-constrained Federated Learning. However, several critical trade-offs and hardware-specific behaviors warrant further analysis. The proposed pipeline is shown to be a robust framework for resource-constrained FL, but with several critical trade-offs and hardware-specific behaviors.

6.1 Knowledge Distillation

Our results in Stage 1 show that standard methods such as PTQ and unstructured pruning, while immediately effective, are limited by the architecture. Quantization provides a fixed 4× reduction, and unstructured pruning in standard deep learning frameworks often fails to result in an actual speedup due to dense-matrix masking. Knowledge Distillation (KD) targets the architecture itself, solving this limitation.

Distilling ResNet-18 into BiggerStudent resulted in a 38× size reduction and a 5× latency drop with a small accuracy cost. This shows that for edge applications, designing a student model that matches hardware constraints may be more effective than relying on the common model compression methods. The difficulty with Knowledge Distillation, however, is that it's not as straightforward as PTQ or pruning: it requires a strong model definition and hyperparameter tuning.

6.2 Federated Scalability and the Fusion Gap

The 99.60% drop in communication bandwidth when using FedDF is extremely useful in real-world scenarios, as in many cases transmitting gigabytes of data like in FedAvg would not be possible.

The tradeoff for the 99.60% drop is the large, 7.1% drop in server model's accuracy. According to the right plot of Figure 3, the client performance remains high, which implies that the accuracy drop is not caused by the failure of models to learn. The problem most likely lies in the server stage of FedDF - distilling knowledge from the ensemble of models. The soft client logits contain lossy dark knowledge, and as clients become more diverse, distillation becomes more difficult. We define this accuracy drop caused by problems with distilling knowledge from the client models as the Fusion Gap.

7 Conclusion

This paper defines a multi-stage model compression pipeline integrating Response-Based Knowledge Distillation, Adaptive Pruning, and INT8 Post-Training Quantization within a Federated Distillation Fusion framework. Experiment results show that this approach can dramatically reduce communication cost, latency and energy footprint on a CPU. While the fusion gap exists, the gains in bandwidth and accuracy make this pipeline applicable to many edge deployment scenarios. In conclusion, Knowledge Distillation provides the flexibility of architecture needed to maximize the strength of traditional model compression techniques and to effectively perform federated learning in resource-constrained scenarios.

8 Future Work

Future research should rework and refine the server-side fusion algorithm, possibly by defining a better loss function or by utilizing the proxy set in more ways. Evaluation of the pipeline under non-IID data conditions will show how robust it would truly be in real-world FL scenarios.

9 Disclosure of LLM use

AI assistance was utilized for proofreading and grammatical refinement of the text, generating Python scripts for visualizations, and debugging of the federated learning simulation code. The final conceptual design, experimental execution, and derivation of conclusions remain solely the responsibility of the author.

References

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics* (A. Singh and J. Zhu, eds.), vol. 54 of *Proceedings of Machine Learning Research*, pp. 1273–1282, PMLR, 20–22 Apr 2017.

[2] Z. Li, H. Li, and L. Meng, "Model compression for deep neural networks: A survey," *Computers*, vol. 12, no. 3, 2023.

[3] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both weights and connections for efficient neural network," in *Advances in Neural Information Processing Systems* (C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, eds.), vol. 28, Curran Associates, Inc., 2015.

[4] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," 2017.

[5] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015.

[6] A. Krizhevsky, "Learning multiple layers of features from tiny images," 2009.

[7] T. Lin, L. Kong, S. U. Stich, and M. Jaggi, "Ensemble distillation for robust model fusion in federated learning," 2021.